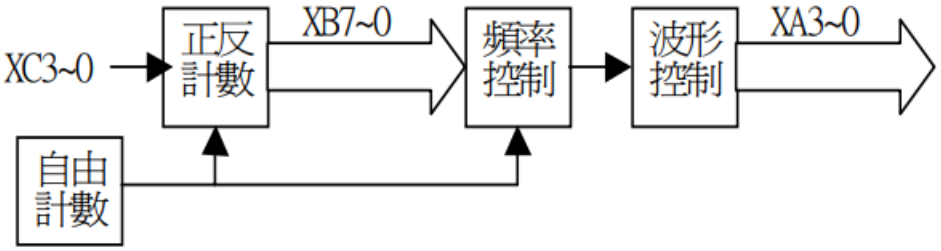
FPGA 數位邏輯系統設計 作業八

第八章 計時處理

通訊 4A 0106C033 桑瑋佑

一 實驗目的:

結合計時電路、除頻電路和波形控制，建立由按鍵控制的可程式頻率產生器。



二 步驟:

實驗時， 先用 LoadPGA 程式將 ch08.rbt 的電路檔載入 FPGA 中，試著依定義來控制按鍵，再由 LED 的顯示狀況去確認 FPGA 是否正確執行。
其中四個 按鍵訊號 分別控制:
XC(0)：計時輸入，按鍵按下時計時器連續加一或減一。
XC(1)：正反控制，當按下時選擇下數，放開則選擇上數。
XC(2)：相位控制，當按下時輸出波形相位變成反向。
XC(3)：計數歸零，當按下時計數值歸零。

在顯示部分:

XA(0)：A 相輸出波形
XA(1)：B 相輸出波形
XA(2)：脈衝輸出波形
XA(3)：方波輸出波形

三 結果:以下為程式碼，為 Ch08.vhd 檔

```
--The IEEE standard 1164 package, declares std_logic, rising_edge(), etc.
library IEEE;
```

```
use IEEE.std_logic_1164.all;
use IEEE.std_logic_arith.all;
use IEEE.std_logic_unsigned.all;

--
--*******************************************************
-- CH08 : practise TIMER and FREQUENCY GENERATOR
--                              designed by Pei-Chong Tang, Jan. 1999
--*******************************************************
--
entity CH08 is
        port
        (       CLK : in    STD_LOGIC;
                XA    : out STD_LOGIC_VECTOR (3 downto 0);
                XB    : out STD_LOGIC_VECTOR (7 downto 0);
                XC    : in    STD_LOGIC_VECTOR (3 downto 0)
        );
end CH08;

architecture CH08_ARCH of CH08 is
        signal SAMPLE        : STD_LOGIC;
        signal FOUT,DOUT : STD_LOGIC;
        signal VALUE         : STD_LOGIC_VECTOR (7 downto 0);
begin
--
--*******************************************************
--
-- 20-BIT free counter
--
FREE_COUNTER : block
        signal Q      : STD_LOGIC_VECTOR (19 downto 0);
        signal DLY : STD_LOGIC;
begin
        process (CLK)
        begin
                if CLK'event and CLK='1' then        --CLK rising
                        DLY <= Q(19);                        --delay
                        Q      <= Q+1;                          --UP counter
```

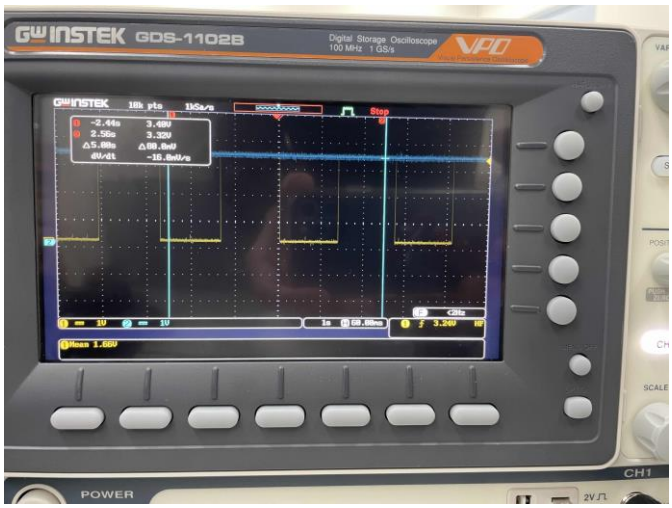
```
            end if;
        end process;
        SAMPLE <= Q(19) and not DLY;                --differential
end block FREE_COUNTER;
--
--*****************************************************
--
-- 8-BIT UP/DOWN parameter setting
--
UPDOWN_COUNTER : block
        signal Q : STD_LOGIC_VECTOR (7 downto 0);
        signal RST,EC,DIR : STD_LOGIC;
begin
        process (CLK,RST)
        begin
            if RST='1' then                         --counter reset
                Q <= "00000000";                    --Q=0
            elsif CLK'event and CLK='1' then --CLK rising
                if EC='1' then                      --CLK enable
                    if DIR='0' then                 --UP/DOWN direction
                        Q <= Q+1;                   --UP     counter
                    else
                        Q <= Q-1;                   --DOWN counter
                    end if;
                end if;
            end if;
        end process;
        RST <= not XC(3);                           --RST signal
        DIR <= not XC(1);                           --DIR signal
        EC    <= SAMPLE and not XC(0);              --EC    signal
        XB    <= not Q;                             --output signal
        VALUE <=      Q;                            --parameter value
end block UPDOWN_COUNTER;
--
--*****************************************************
-- programmable frequency generation
--
FREQ_GENERATION : block
```

```
        signal Q           : STD_LOGIC_VECTOR (8 downto 0);
        signal DLY,EC : STD_LOGIC;
begin
        process (CLK)
        begin
                if CLK'event and CLK='1' then           --CLK rising
                        DLY <= Q(8);
                        if EC='1' then                          --CLK enable
                                Q <= Q + ('0' & VALUE);    --integration
                        end if;
                end if;
        end process;
        EC      <= SAMPLE;
        DOUT <= Q(8) xor DLY;                                   --differential
        FOUT <= Q(8);
end block FREQ_GENERATION;
--
--*******************************************************
-- A/B phase generation
--
PHASE_GENERATION : block
        signal EC,DIR,A,B : STD_LOGIC;
begin
        process (CLK)
        begin
                if CLK'event and CLK='1' then        --CLK rising
                        if EC='1' then                        --CLK enable
                                A <= DIR xor         B;       --A phase generation
                                B <= DIR xor not A;       --B phase generation
                        end if;
                end if;
        end process;
        EC    <= DOUT;                                          --base frequency
        DIR <= XC(2);                                          --direction control
        XA(0) <= A;                                            --A-phase output
        XA(1) <= B;                                            --B-phase output
        XA(2) <= not DOUT;                                     --pulse    output
        XA(3) <=         FOUT;                                  --square output
```

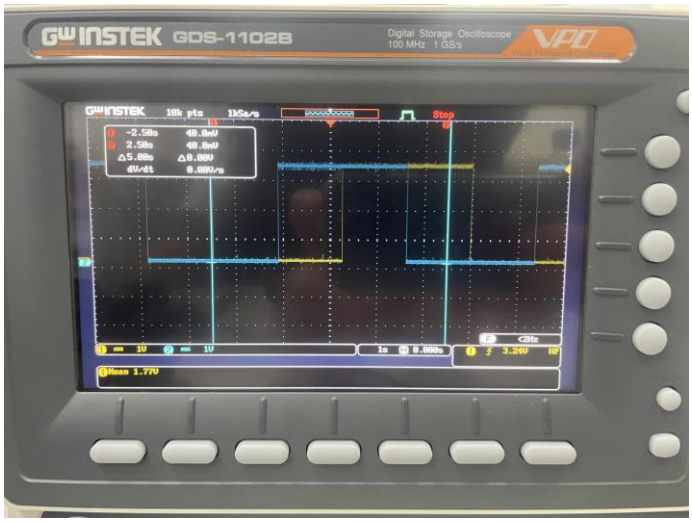
```
end block PHASE_GENERATION;
--
--*****************************************************
-- end of architechture
--
end CH08_ARCH;
```

以下為成果圖:

此為將示波器連接到 XA(0)和 XA(1)，示波器上顯示 AB 兩相位的波型，Ch1 為 B，Ch2 為 A，Ch1 超前 Ch2 九十度。



此為示波器連接到 XA(2)和 XA(3)，示波器上顯示脈衝波形和方波波形，Ch1 為方波，Ch2 為脈衝，可惜實驗過程中無法顯示出脈衝波。



此為按下 XC(3)再將示波器連接到 XA(0)和 XA(1)，由於按下相位控制，A、B 兩相位相反，變成 A 超前 B 九十度，Ch1 為 B，Ch2 為 A。

四 重點提醒

1. 積分除頻器的頻率分布相對均勻，但輸出的信號會有不均勻性。若只需考慮平均頻率，積分除頻器是個不錯的選擇。

2. AB 相式設計具備良好的抗噪聲性能。雖然一開始看起來有些複雜，但透過真值表分析後，會發現其實比預期更簡單。

3. 由於輸入信號直接控制 SET 信號，而 SET 信號是非同步的，且其優先級高於 CLK 時鐘信號，因此單擊電路可能會有延遲時間問題。通過對 SET 信號進行分段處理可以解決此問題，但需要使用後緣微分，主要是為了防止微分後的 SET 信號與下一個 CLK 前緣重疊。

五 心得結論

難度的確提升了許多，老師也投入更多時間來講解。我們學到了各種方法，也能理解不同方法在使用上的差異和優劣。例如，先前學

過使用自由計數器來處理頻率，但為了應對更多頻率需求，我們進一步學習到上下數除頻電路來實現 N 倍分頻，若要達到更均勻的分頻，則可用積分除頻電路。學得越多，解決問題時的選擇也更多。課程的最後會講到波形產生，從基本的脈衝和方波開始，搭配雙向型式能降低噪聲干擾。學習內容增多，需要更多的複習和練習，才能真正掌握並靈活運用。